

Smart Turf Booking & Management System

CRUD OPERATIONS DOCUMENTATION

Project	Smart Turf Booking & Management System
Database	MySQL
Backend	Spring Boot REST API
Frontend	HTML / Thymeleaf Web Application
API Base URL	http://localhost:8080
Database Tables	8
CRUD Scope	Users, Turfs, Turf Slots, Bookings, Payments, Reviews, Complaints, Notifications

Architecture

Frontend
↓
HTML / Thymeleaf
↓
HTTP / REST
↓
Spring Boot Controller
↓
Service Layer
↓
Spring Data JPA Repository
↓
MySQL

Purpose: This document records the Create, Read, Update and Delete operations for the Smart Turf Booking application, together with REST endpoints, database mapping and booking-specific business operations.

Reference: The supplied LicenseGuard document is organized around CRUD overview, REST conventions, module operations, SQL reference, data flow, validation, verification and final summary. This document follows that structure for Smart Turf Booking. ■filecite■turn2file0■L29-L54■

1. CRUD Overview

CRUD stands for Create, Read, Update and Delete. Smart Turf Booking uses Spring Boot and Spring Data JPA to communicate with MySQL, while the web frontend communicates with the backend through HTTP.

Entity / Module	Create	Read	Update	Delete / Special
User	POST	GET	PUT	DELETE / deactivate
Turf	POST	GET	PUT	DELETE / deactivate
Turf Slot	POST	GET	PUT	DELETE
Booking	POST	GET	PUT	DELETE / cancel
Payment	POST	GET	—	DELETE where allowed
Review	POST	GET	PUT	DELETE
Complaint	POST	GET	PUT	DELETE
Notification	POST	GET	PUT	DELETE / mark read

2. REST API Convention

HTTP Method	CRUD Meaning	Typical Use
POST	Create	Create a new record
GET	Read	Retrieve one or more records
PUT	Update	Modify an existing record or state
DELETE	Delete	Remove a record where deletion is allowed

Typical HTTP results: 200 OK, 201 Created, 400 Bad Request, 404 Not Found and 409 Conflict.

3. User CRUD Operations

The users table stores customers and administrators.

Operation	REST API	Purpose
Create	POST /api/users	Create user
Read all	GET /api/users	List users
Read one	GET /api/users/{id}	Retrieve user
Update	PUT /api/users/{id}	Modify user
Delete	DELETE /api/users/{id}	Delete/deactivate

Database-level SQL examples:

```
SELECT * FROM users;
INSERT INTO users (name,email,password,phone,role,status) VALUES
('Rahul','rahul@gmail.com','12345','9876543210','CUSTOMER','ACTIVE');
UPDATE users SET phone='9999999999' WHERE user_id=1;
UPDATE users SET status='INACTIVE' WHERE user_id=1;
```

4. Turf CRUD Operations

The turfs table stores turf/ground details such as name, location, description, hourly price and status.

Operation	REST API	Purpose
Create	POST /api/turfs	Create turf
Read all	GET /api/turfs	List turfs
Read one	GET /api/turfs/{id}	Retrieve turf
Update	PUT /api/turfs/{id}	Modify turf
Delete	DELETE /api/turfs/{id}	Delete/deactivate

Database-level SQL examples:

```
SELECT * FROM turfs;
INSERT INTO turfs (turf_name,location,description,price_per_hour,status) VALUES ('Green
Turf','Thane','Football turf',800.00,'ACTIVE');
UPDATE turfs SET price_per_hour=900 WHERE turf_id=1;
UPDATE turfs SET status='INACTIVE' WHERE turf_id=1;
```

5. Turf Slot CRUD Operations

The turf_slots table stores time slots belonging to a turf.

Operation	REST API	Purpose
Create	POST /api/turf-slots	Create slot
Read all	GET /api/turf-slots	List slots
Read by turf	GET /api/turf-slots?turfId={id}	List turf slots
Update	PUT /api/turf-slots/{id}	Modify slot
Delete	DELETE /api/turf-slots/{id}	Delete slot

Database-level SQL examples:

```
SELECT * FROM turf_slots;
SELECT * FROM turf_slots WHERE turf_id=1;
INSERT INTO turf_slots (turf_id,start_time,end_time,status) VALUES (1,'18:00:00','19:00:00','AVAILABLE');
UPDATE turf_slots SET status='UNAVAILABLE' WHERE slot_id=1;
```

6. Booking CRUD Operations

Bookings record customer reservations. Booking is more than basic CRUD because availability and duplicate-booking rules must be checked.

Operation	REST API	Purpose
Create / Book	POST /api/bookings	Create after availability check
Read all	GET /api/bookings	List bookings
Read by user	GET /api/bookings?userId={id}	Customer bookings
Read by turf	GET /api/bookings?turfId={id}	Turf bookings
Update	PUT /api/bookings/{id}	Supported state change
Cancel	PUT /api/bookings/{id}/cancel	Cancel booking

Database-level SQL examples:

```
SELECT * FROM bookings;
SELECT * FROM bookings WHERE user_id=1;
UPDATE bookings SET status='CANCELLED' WHERE booking_id=1;
```

Business rules: the turf and slot must exist; the selected slot must be available for the requested date/time; duplicate active bookings must be rejected; cancellation should change the booking state so history can be preserved.

7. Payment Operations

Payments store payment records related to bookings. A college-project implementation can use payment simulation.

Operation	REST API	Purpose
Create / Pay	POST /api/payments	Create payment
Read all	GET /api/payments	List payments
Read one	GET /api/payments/{id}	Retrieve payment
Read by booking	GET /api/payments?bookingId={id}	Payment for booking

Database-level SQL examples:

```
SELECT * FROM payments;  
INSERT INTO payments (booking_id,amount,payment_method,status) VALUES (1,800.00,'UPI','SUCCESS');
```

8. Review Operations

Reviews store customer feedback for a turf.

Operation	REST API	Purpose
Create	POST /api/reviews	Create review
Read	GET /api/reviews	List reviews
Read by turf	GET /api/reviews?turfId={id}	Turf reviews
Update	PUT /api/reviews/{id}	Modify review
Delete	DELETE /api/reviews/{id}	Delete review

Database-level SQL examples:

```
SELECT * FROM reviews;  
SELECT * FROM reviews WHERE turf_id=1;  
UPDATE reviews SET rating=5,comment='Excellent turf' WHERE review_id=1;
```

9. Complaint Operations

Complaints store customer issues and their status.

Operation	REST API	Purpose
Create	POST /api/complaints	Submit complaint
Read	GET /api/complaints	List complaints
Read by user	GET /api/complaints?userId={id}	Customer complaints
Update	PUT /api/complaints/{id}	Update complaint
Delete	DELETE /api/complaints/{id}	Delete complaint

Database-level SQL examples:

```
SELECT * FROM complaints;  
UPDATE complaints SET status='RESOLVED' WHERE complaint_id=1;
```

10. Notification Operations

Notifications store booking, payment and system messages.

Operation	REST API	Purpose
Create	POST /api/notifications	Create notification
Read	GET /api/notifications	List notifications
Read by user	GET /api/notifications?userId={id}	User notifications
Update	PUT /api/notifications/{id}/read	Mark read
Delete	DELETE /api/notifications/{id}	Delete notification

Database-level SQL examples:

```
SELECT * FROM notifications;  
UPDATE notifications SET is_read=1 WHERE notification_id=1;
```

11. CRUD Data Flow

The web frontend remains separate from database persistence. Requests flow through the Spring Boot backend.

User
↓
HTML / Thymeleaf Frontend
↓
HTTP Request
↓
Spring Boot Controller
↓
Service Layer
↓
Spring Data JPA Repository
↓
MySQL Database

Layer	Responsibility
Web Frontend	Forms, tables, validation hints, navigation and displaying results.
Controller	Receives HTTP requests and returns views or API responses.
Service	Implements business rules and transactions.
Repository	Performs persistence through JPA/Hibernate.
MySQL	Stores the eight project tables and relationships.

12. CRUD Validation and Error Handling

HTTP	Meaning
200 OK	Successful GET/PUT
201 Created	Successful POST
400 Bad Request	Invalid request, validation failure or invalid booking operation
404 Not Found	Requested entity does not exist
409 Conflict	Duplicate resource or unavailable slot
500 Internal Server Error	Unexpected server error

Recommended validation includes required fields, valid email, positive turf price, valid slot times, valid booking date, availability checking, duplicate-booking protection and safe status transitions.

13. SQL CRUD Reference

CREATE

```
INSERT INTO turfs (turf_name, location, description, price_per_hour, status)
VALUES ('Green Turf', 'Thane', 'Football turf', 800.00, 'ACTIVE');
```

READ

```
SELECT * FROM turfs;
SELECT * FROM turf_slots WHERE turf_id=1;
SELECT * FROM bookings WHERE user_id=1;
```

UPDATE

```
UPDATE turfs SET price_per_hour=900 WHERE turf_id=1;
UPDATE bookings SET status='CANCELLED' WHERE booking_id=1;
```

DELETE / DEACTIVATE

```
UPDATE turfs SET status='INACTIVE' WHERE turf_id=1;
DELETE FROM reviews WHERE review_id=1;
```

For important business records such as bookings and payments, status-based cancellation/deactivation is preferable when history must be preserved.

14. Database Relationship Support

Relationship	Cardinality	CRUD Impact
USER → BOOKING	1 : N	Customer can have many bookings.
TURF → TURF_SLOT	1 : N	Turf can have many slots.
TURF → BOOKING	1 : N	Turf can have many bookings over time.
BOOKING → PAYMENT	1 : N / project-dependent	Payments are associated with bookings.
USER → REVIEW	1 : N	Customer can create reviews subject to rules.
TURF → REVIEW	1 : N	Turf can receive many reviews.
USER → COMPLAINT	1 : N	Customer can submit complaints.
USER → NOTIFICATION	1 : N	User can receive notifications.

15. CRUD Implementation Verification

Verification Item	Expected Result
Spring Boot startup	PASS after application starts without errors
MySQL connection	PASS after datasource connects
JPA/Hibernate initialization	PASS after entities map successfully
Users CRUD	Create/read/update/delete or deactivate
Turfs CRUD	Create/read/update/delete or deactivate
Turf Slots CRUD	Create/read/update/delete
Booking operations	Availability and cancellation rules work
Payment	Simulation and booking linkage work
Reviews	Create/read/update/delete
Complaints	Create/read/update/delete
Notifications	Create/read/mark-read/delete
Validation	Invalid requests return appropriate errors

16. Final CRUD Summary

Module	CRUD Status	Key Feature
Users	Complete	Customer/admin management
Turfs	Complete	Turf master-data management
Turf Slots	Complete	Time-slot availability
Bookings	CRUD + business operations	Availability and cancellation
Payments	Create/Read	Payment simulation
Reviews	Complete	Ratings and feedback
Complaints	Complete	Issue tracking
Notifications	Complete	System messages

Final architecture

Frontend
↓
HTML / Thymeleaf
↓
Spring Boot Controller
↓
Service Layer
↓
Spring Data JPA
↓
MySQL

Conclusion: Smart Turf Booking implements CRUD through Spring Boot, Spring Data JPA and MySQL. Domain-specific operations such as slot availability, booking cancellation and payment confirmation are handled in the service layer.

Source structure: adapted from the supplied LicenseGuard CRUD document, which covers CRUD overview, module endpoints, SQL reference, data flow, validation, verification and final summary. ■filecite■turn2file0■L268-L319■